UNIVERSIDAD NACIONAL DE MISIONES

Facultad de Ingeniería OBERA

ARQUITECTURA DE COMPUTADORAS

Laboratorio N° 3c

Tema: Desarrollo del Benchmark

Objetivo general

Para el desarrollo del laboratorio 3, en esta etapa cada grupo deberá implementar una función específica en *assembly* de la ISA LC-3, la cual puede ser parte de un algoritmo utilizado para implementar una de las funciones elementales (SQRT, LOG, EXP, SIN, COS), o una funcionalidad de la calculadora, como por ejemplo, pasar de decimal a hexadecimal o binario. Estos códigos podrán ser desarrollado mediante programación utilizando LLMs (*Large Language Models*, Modelos Grandes de Lenguaje), desde el uso de herramientas que tienen de interfaz chatbots como ChatGPT, Deepseek, Gemini, Claude, etc.; hasta programación agéntica, la cual incorpora más funcionalidades de razonamiento, planificación y depuración.

Todos los grupos deberán aportar sus códigos en *assembly* (lo que sería parte de la carga de trabajo, *workload*, de la calculadora) para poder integrarse en un solo *benchmark*, el cual se utilizará en la etapa final para diseñar la nueva ISA.

De objetivo de máxima las funcionalidades desarrolladas deberán utilizarse para integrarse en funciones elementales (SQRT, LOG, etc.), y de mínima solamente formar un banco de algoritmos que deberán funcionar independientemente de llegar a realizar dichas funciones. Esto se debe a que el tiempo que se tiene para hacer el laboratorio puede ser insuficiente para llegar a cumplir el objetivo de máxima, ya que requiere de coordinación entre distintos grupos para poder poner a funcionar las funciones elementales, a partir de los códigos de cada grupo.

En la etapa de análisis de los algoritmos que deberán implementar los grupos, deberán escribir o generar en lenguajes de alto nivel los programas que permiten implementarlos, para a partir de estos depurarlos, estudiarlos y entenderlos antes de pasar a *assembly* de la LC-3.

Objetivos específicos

Se espera que los estudiantes alcancen los siguientes objetivos durante el desarrollo del laboratorio

- **Comprender los alcances y limitaciones del uso actual de la Inteligencia Artificial Generativa en el desarrollo de soluciones de software, particularmente en la generación de código en lenguajes de alto nivel para el desarrollo de una**

carga de trabajo, la cual formará parte del *benchmark* para la siguiente etapa del laboratorio (desarrollo del simulador funcional).

- **Conocer los problemas de seguridad que tienen en la actualidad los LLM cuando se los utiliza en comportamientos agenticos.**
- **Comprender los algoritmos utilizados en computación de funciones matemáticas en formatos de punto fijo.**
- **Depurar y corregir errores en programas escritos en lenguajes de alto nivel y en *assembly*.**

Temas para cada grupo

A continuación, se presenta un listado de todos los temas a asignar a los distintos grupos:

Multiplicación	por CORDIC
Multiplicación	por algoritmo the shift + sumas
Multiplicación	algoritmo de Booth
División	por método división larga
División	Por LUT+Reducción de rango e interpolación por Newton - Raphson
Transformación Bilineal	Explicar que rol juega en el Algoritmo de CORDIC esta parte e implementar la división por CORDIC
Parte de CORDIC	reducción de argumento Manual
Parte de CORDIC para LOG	Iteración de la serie atanh
SQRT	non restoring
SQRT	por Newton-Raphson método raíz cuadrada reciproca
LOG	Borchardt (requiere tabla para SQRT)
EXP (e^x)	Algoritmo multiplicativo de alta radix
Q3.12->Q7.8	usando multiplicación por suma sucesivas y corrimiento
DEC->BIN->HEX	a partir de un número ingresado en decimal pasar a binario y hexa
Mayor denominador común + Mínimo común múltiplo	
POW (x^y)	Multiplicación sucesiva (la multiplicación resolver por suma sucesivas)
POW (x^y)	exponenciación binaria (la multiplicación resolver por suma sucesivas)
interfaz para el banco de algoritmos / calculadora	debe poder dar formato de decimal a binario con punto fijo y cambiar entre modo entrada por teclado y pasaje por memoria los datos para calcular los resultados de cada algoritmo
SIN	por reducción + Taylor de 4 términos (la multiplicación resolver por suma sucesivas).

SIN and COS por tabla	se debe tener una sola tabla pero para la otra función aplicar un desplazamiento de pi/2.
SIN	por Chebyshev y Padé. (la multiplicación resolver por suma sucesivas)
Simulador Funcional com interfaz web	capacidad de subir cada grupo su archivo y hacer una interfaz con .EXTERNAL

A los grupos que les haya tocado trabajar con funciones que requieren formato en punto fijo, deberán explicitar que formatos usan, cual es el rango de la variable de entrada y de salida, y la precisión alcanzada.

Cada grupo deberá realizar análisis de su algoritmo en un lenguaje de alto nivel, usar el depurador para ayudarle a entender el funcionamiento, y luego recién tratar de implementar en *assembly*.

En una semana desde que se dan los temas los grupos deberán organizarse para presentar los siguientes avances, todos realizados sobre el análisis en el programa escrito en lenguaje de alto nivel:

- Funcionamiento del algoritmo, que objetivo tiene, un breve comentario de quien lo propuso en caso de que se pueda obtener esta información.
- Número de iteraciones que les lleva al algoritmo para obtener el resultado.
- Que formato de punto fijo debe utilizar, rango alcanzado (aquí no solamente es para las variables de entradas sino también para la salida que se puede obtener) y precisión mínima y máxima.
- La organización de los grupos debe regirse por el principio de “trabajar problemas en común”, por ejemplo, quienes estén realizando rutinas de multiplicación, deben coordinarse entre dichos grupos para exponer sus resultados en un tiempo no superior a 15 minutos (aquí si logran presentar en menos tiempo, si es que amerita, lo pueden hacer).

Para la presentación de estos avances no es necesario tener las rutinas funcionando en *assembly*.

Cuando comiencen con el código en *assembly*, también deberán implementar depuración y encontrar así errores que puedan haber, repararlos e integrar su código al de los otros grupos.

Para la integración se plantearán diferentes alternativas, estando entre estas la coordinación de los grupos a través de Git y GitHub, o la incorporación de las rutinas en

assembly a un simulador web que les permita lograr la funcionalidad de todo el *benchmark* (posiblemente el grupo 8 lleve esta tarea adelante).

Definición de tutores para cada grupo

Los grupos tendrán asignados un tutor a quien deberán dirigir consultas que puedan surgir sobre los algoritmos que deben implementar y su desarrollo. A continuación, se detallan los tutores para cada grupo:

Dr. Ing. Aldo Benitez: Grupos 1, 6, 10, 22.

Ing. Roberto Nicolás Schuster: Grupos 4, 20, 24, 30.

Ing. Patricia Feltan: Grupos 5, 12, 19, 23.

Ing. Gustavo Rossler: Grupos 11, 15, 27.

Ing. Eduardo Díaz: Grupos 7, 8, 14.

Dr. Ing. Roberto Carballo: Grupos: 3, 9, 18, 21.

Cada tutor deberá realizar un seguimiento de las actividades que van llevando adelante los grupos, registrarlas para el conocimiento de todos los docentes en una planilla compartida.

Sobre el uso de las herramientas de IA generativas

Se recomienda para el desarrollo de los algoritmos, tanto en lenguaje de alto nivel como en *assembly*, hacer un primer intento con una IA generativa para tener la estructura inicial armada. Mediante el uso de estas herramientas es posible agilizar significativamente el análisis y estudio de los algoritmos, siempre y cuando lo generado (tanto el código como las explicaciones que se puedan obtener de la misma herramienta) sirva comprender el principio de funcionamiento.

Hay que tener en cuenta que dependiendo la forma en que se utilicen los LLM, es posible incurrir en problemas de seguridad informática. A continuación, se describen los cuidados a tener en cuenta.

En primer lugar se presenta una escala de riesgos de acuerdo a los métodos de uso actuales:

Herramientas Web ----------> Agéntico Autónomo (Uso local)

Herramientas Web

El uso es a través de aplicaciones web, como ChatGPT (https://chatgpt.com/), Claude (https://claude.ai), Gemini (https://gemini.google.com/app), mantiene los riesgos de exposición de datos privados al mínimo, ya que al correr en servidores en la nube, salvo que uno introduzca datos privados a través del chat, los modelos no tienen como extraer información no autorizada del usuario. Por lo tanto, como regla general se recomiendan las siguientes pautas para un uso seguro (fuente: https://www.eset.com/blog/en/home-topics/cybersecurity-protection/is-chatgpt-safe-2026-guide/):

La "Lista Roja": Información que nunca deberías compartir

Tratá a cada chat como si pudiera hacerse público. Nunca ingreses:

- **Identificadores personales:** Números de documento, números de pasaporte, direcciones, etc.
- **Información financiera:** Números de tarjeta, datos de cuentas bancarias, identificaciones fiscales (CUIT/CUIL), etc.
- **Contraseñas, claves de API** o cualquier otro secreto (por ejemplo, tokens de autenticación de dos factores).
- **Datos de empresas:** Código fuente, listas de clientes, documentos internos, informes financieros no públicos, documentos legales.
- **Información de salud:** Cualquier dato protegido por leyes de privacidad médica o regulaciones similares.

La lista es corta porque la regla que la respalda es simple: SI NO SE LO ENVIARÍAS POR CORREO ELECTRÓNICO A UN EXTRAÑO, NO LO ESCRIBAS EN UN CHATBOT.

Modo agéntico local

Cuando se trata del uso de los LLM en nuestra computadora personal, ya sea corriendo el modelo de manera local o a través de servicios web pero que se conectan a

nuestra computadora a través de una IDE (*Integrated Development Enviroment*), como por ejemplo VS code, Cursor, Antigravity, o alguno similar; el riesgo pasa a ser elevado, ya que los LLM pueden comportarse en forma no deseada por el usuario y al tener control sobre nuestro dispositivo (terminal, navegador web, etc.), pueden provocar desde la modificación de archivos hasta su eliminación, exposición de los datos del navegador, o introducción de código malicioso. A continuación, se enumeran los casos más relevantes:

1. **Agencia Excesiva y Ejecución Descontrolada en Agentes**: Herramientas agénticas que ejecutan comandos de terminal o modifican múltiples archivos pueden causar borrados de datos, instalación de dependencias no auditadas o alteración de configuraciones críticas si no existe supervisión humana constante.

2. **Inyección Indirecta de Prompts vía Repositorio**: Un atacante puede colocar instrucciones maliciosas en comentarios de código, *issues* o archivos de documentación. Cuando el agente de IA lee el contexto del proyecto, estas instrucciones pueden manipularlo para insertar puertas traseras (*backdoors*) o filtrar claves de acceso (ver por ejemplo GitLost: https://noma.security/noma-labs/gitlost-how-we-tricked-githubs-ai-agent-into-leaking-private-repos).

3. **Ceguera Contextual y Falsa Confianza ("Vibe Coding")**: El código generado luce sintácticamente impecable y pasa pruebas sencillas, pero omite el manejo de excepciones, condicionales límite (*edge cases*) y concurrencia, lo que lleva a los desarrolladores a aceptarlo sin una revisión de código rigurosa.

4. **Exposición del historial de navegación y otros datos privados que se puedan encontrar en la computadora**

   Los mecanismos de protección a nivel de software (*prompt guardrails*) que tienen incorporado algunos IDE son frágiles. Aunque el modelo esté programado para no realizar acciones dañinas, basta una ligera manipulación en la instrucción (*prompt engineering*) para que examine el historial de navegación y extraiga información sensible (ver por ejemplo https://youtu.be/OE_lLNCNfQo?si=bIKcm6Yf4R_DDl9W).

Establecido los riesgos que se tienen, a continuación, se detallan ciertas medidas a tener en cuenta para mitigar los mismos:

- Realizar un respaldo de los datos de nuestra computadora, para en caso de que se eliminen archivos importantes se tenga como recuperarlos.
- No darle la mayor cantidad de permisos a los modelos. Existen configuraciones dentro de los IDE que permiten supervisar las acciones que

la IA toma, donde el usuario puede aceptar o rechazar para evitar fallos o mal funcionamiento.

- Eliminar el historial de navegación luego de hacer uso de cuentas bancarias o cualquier aplicación que pueda requerir nuestros datos privados.
- Si no es estrictamente necesario, no comprometer llaves de acceso, API o datos.
- Si se requieren mayores medidas para aislar a los agentes, incorporar *sandboxing*, ya sea mediante máquinas virtuales o micro VM (micro virtual machines), que aíslan el *kernel* de ejecución de nuestro sistema operativo del que utiliza el agente.

Como corolario sobre este tema de seguridad cuando se hace uso de estas herramientas, para el nivel de tareas que se pretenden llevar adelante en este laboratorio, ambos modos de funcionamiento de los LLM, , tanto el uso web como local, pueden traerle beneficios para desarrollar los códigos, sin embargo, es prudente en caso de usar los agentes integrados a IDEs, supervisar de cerca la actividad de los mismos para evitar sobre todo el borrado de archivos que puedan resultar importantes y la instalación de paquetes sospechosos.

Sobre el control de versiones

Para tener un mejor seguimiento de los programas que van desarrollando, se recomienda independientemente de usar IA generativa o no, utilizar mecanismos de control de versiones como Git, el cual permitirá trabajar en forma organizada sobre los archivos en desarrollo, además de poder volver hacia atrás en caso que alguna actualización del trabajo no consigue el funcionamiento deseado.